ChatGPT

Nested Learning 理论深度分析及 nanochat 模型改造方案

嵌套学习（Nested Learning）的核心思想与传统深度学习区别

嵌套学习（Nested Learning，NL）是一种将模型视为一系列**嵌套的、多级的优化问题**的新范式[1]。每个子问题有自己独立的"**上下文流**"（即该组件所看到的输入、梯度或状态序列）和**更新频率**[2][3]。这意味着，与传统深度学习把模型架构和优化过程割裂开的做法不同，NL 将**模型结构与优化算法统一建模**为同一系统中不同时间尺度的嵌套循环[4][3]。具体来说，频繁更新的参数属于模型的内层（短时尺度），更新较慢的参数构成外层（长时尺度），各层按更新频率形成层次结构[5]。**每一层模块都会将其自身的上下文流"压缩"进参数中**，实现对不同时间尺度信息的记忆[6][7]。相比之下，传统深度学习通常是固定多层网络+单一优化器的扁平训练流程，模型各层同时更新，无法显式地区分快慢记忆层[8]。NL 提供了一种更加**多层次、白盒化**的视角来解释模型如何从数据中学习，在这一视角下，**反向传播、本地注意力乃至优化器本身**都可以被统一解释为特殊的"记忆模块"[9][10]。这种重新表征使我们能够识别现有深度学习方法实际上是通过**压缩自身的上下文**来从数据学习，并揭示大模型中**in-context learning**（上下文学习）能力的来源[11][12]。

换言之，嵌套学习将**架构和优化过程视作一个整体**：模型被刻画为由多个具有各自学习过程的子模块组成的层次系统，每个模块在不同的时间尺度上适应数据[4][13]。这种理念受启发于人脑的多时间尺度学习机制，例如人类神经系统中既有快速突触可塑性（短期变化）又有慢速结构重组（长期变化）[14]。因此，与传统深度学习需要在训练新任务时面临遗忘旧知识的权衡不同，NL 通过在模型内部预先设计不同速率的学习组件，可**直接针对灾难性遗忘问题**进行优化[15]。总的来说，NL 理论的核心在于：**模型 = 多层嵌套优化**，各层有独立的上下文和更新规则，这一框架为设计更**高阶的上下文学习**能力提供了新途径[12][13]。

优化器视为嵌套记忆系统的理解与改造

NL 框架下的一个重要观点是：**优化器本身可以被视为一种记忆模块**[16][17]。传统梯度优化算法（如带动量的 SGD、Adam 等）在 NL 看来其实是**用于压缩梯度信息的联想记忆**[18][17]。例如，带**动量 SGD**的更新规则可写作：

- $W_{i+1} = W_i + m_{i+1}$
- $m_{i+1} = \alpha \, m_i - \eta \, \nabla L(W_i)$

其中 m_i 是动量项。研究表明，这一动量更新可被解释为对某个内部目标函数执行一次梯度下降，目标是让记忆向量 m 与当前梯度对齐（内积最大）[19][20]。因此，**动量项实际扮演了一个记忆单元的角色，用于将过去的梯度信息压缩存储起来**[21][22]。换句话说，动量 SGD 是一种**线性联想记忆**：它用与梯度的点积相似度作为内部学习目标，产生了一种 Hebb 型的更新规则[17][23]。然而这种简单的内积关联仅利用当前梯度和动量的直接相似，**没有建模不同样本梯度之间的关系**[24]。因此，NL 提示我们可以通过**改进优化器的内部目标**来获得更强的记忆能力[24][25]。

基于此视角，研究者对经典优化器进行了**更深层的记忆改造**[16]。一方面，将动量的**内部相似度目标从点积改为L2回归损失**，促使动量更有效地拟合梯度序列，从而在有限容量下更好地记忆住过去梯度[26][27]。这种改动使动量更

新考虑到了样本梯度的分布差异，**提高了对噪声和样本相关性的鲁棒性** [27] [25]。另一方面，研究者进一步将动量记忆从线性扩展为**非线性神经网络**：例如用一个多层感知机（MLP）来替代简单的线性动量项，从而构建"**深度动量梯度下降**"优化器 [28]。这意味着动量状态由**神经网络记忆模块**产生，可以对梯度进行非线性变换（例如结合Newton-Schulz近似实现二阶信息） [29]。事实上，已有的一些高级优化器（如 MuZero 团队提出的 Muon）可以被看作此框架的特例 [29]。通过这些改造，优化器不再是训练过程外部的固定算法，而成为模型**嵌套体系的一部分**：**拥有深度记忆和自适应学习规则的优化器**能够根据数据特征做出更智能的更新决策 [26] [25]。在工程实现上，这意味着可以设计**自带"记忆单元"的优化器**——例如在 nanochat 中用一个内置神经网络来维护梯度历史——从而增强模型的学习能力和上下文依赖更新能力。

HOPE 架构剖析：连续记忆、模型自修改与多频梯度更新

HOPE（Help Our Prometheus Evolve? 注：原文未明确缩写含义，这里用其名称）架构是NL理念的综合实践，它结合了**连续谱记忆系统**和**自修改的序列模型**，实现了多层次的嵌套学习能力 [30] [31]。HOPE 可以被视为对已有长短期记忆架构 **Titans** 的拓展和升级：Titans 提供了一种在**推理时学习记忆**的机制（会针对"意外"事件动态存储，以便长期依赖建模），但 Titans **仅存在两个参数更新层级**，只能实现**一阶的上下文内学习** [32]。HOPE 在此基础上扩展出两大关键特性： [33]

- **(1) 自我修改（Self-Modifying）**：引入**自引用的学习过程**，使模型能够**在运行时优化自身的记忆参数** [31] [34]。也就是说，HOPE 可以在处理序列数据的同时，通过嵌套的内部优化步骤来更新自己的某些权重，实现类似元学习的效果 [35] [36]。理论上，这提供了**无穷嵌套**的上下文学习层级 —— 模型可以在推理过程中不断调整自身，以适应新的信息 [34] [36]。这类似于"模型在读取文本时一边学习一边改变自己的部分结构"，实现**递归式的自我改进** [36]。

- **(2) 连续记忆系统（Continuum Memory System, CMS）集成**：HOPE 将多个**不同更新频率的记忆模块**融入架构，使模型能够在**更长上下文窗口**上有效工作 [33] [37]。Titans 只有快速和慢速两类记忆（类似短期和长期），而 HOPE 增加了 **多层次的记忆块**，让记忆更新以不同速率发生，形成从快速到缓慢的连续谱 [33] [38]。这种设计使 HOPE 能够捕获 **更加丰富的时间尺度信息**，显著扩展模型处理超长序列和持续学习的能力。

下面分别对 HOPE 中的关键机制进行剖析：

Continuum Memory System：连续谱记忆系统

连续记忆系统（CMS）是 NL 提出的新型记忆架构，它**打破了传统"短期/长期记忆"二元划分**，用一个按更新频率排列的记忆模块序列来覆盖从短时到长时的各个尺度 [39] [40]。具体而言，CMS 定义了一串按频率不同更新的 MLP 子模块，记作 $MLP(f_1), MLP(f_2), ..., MLP(f_k)$ [7]。它们在模型中的连接方式是**顺序级联**：输入经过第一个记忆块处理后，再传给第二个，以此类推，最后得到模块链的输出 [7]。第ℓ级记忆块的参数**每经过 $C^{(\ell)}$ 个时间步才更新一次**（其中 $C^{(\ell)}$ 可理解为该层的更新间隔） [7]。因此，不同记忆层记录和压缩的是**不同时长尺度**的上下文信息：更新频率高的层捕获快速变化的近期信息，而更新频率低的层则逐渐聚合较长时间尺度的模式 [7] [41]。在这种架构下，传统Transformer的记忆机制只是一个特例：Transformer每层只有一个前馈网络作为长期记忆，如果我们将CMS的记忆块数量 k 取为1，就退化回普通Transformer的情形 [42]。相比之下，CMS提供了**短、中、长**多层次的记忆谱系，显著增强了模型持续学习的能力 [40] [43]。

从直观上看，可以把**注意力机制**类比为CMS中的"最短期"记忆（工作记忆）——每次前向传播都重新计算，**即时更新**；**普通前馈层参数**则类似"长期"记忆——在预训练后很少更新，相当于固化了的知识 [44]。CMS引入介于两者之间

的**中间层次记忆模块**：它们的参数在训练过程中**以中等频率更新**，既能随着新数据缓慢调整，又不会像注意力那样每步变动 [7] [45]。这种连续谱设计与神经科学中的**多时间尺度记忆巩固**过程类似：不同脑区以不同速度学习，却共享信息共同构建记忆 [40]。对于持续学习而言，CMS提供了一个机制，允许模型**针对不同时间跨度的信息分别存储**：哪些信息需要**快速记住**，哪些应该**长期保留**，都有对应的模块去处理 [45]。因此，在HOPE架构中集成CMS后，模型在面对超长文本或连续任务时能够**更有效地累计知识且不遗忘** [15] [46]。

自修改模块：Titans架构与自我更新机制

自修改模块指的是HOPE中那部分能够在推理过程中自行学习和调整的组件。本质上，这是对**Titans序列模型**的继承与发展。Titans是一种**长程记忆RNN**架构，它通过一个**可训练的外部记忆模块**在推理时**记忆"惊奇"事件**，从而辅助注意力机制对非常久远的上下文进行关联 [47]。具体而言，Titans在模型中增加了一个慢速更新的记忆，当遇到罕见或难预测的输入时将其写入记忆，以便稍后查询，从而**"让 token 几乎可以永远存在"**（即避免重要信息被遗忘） [48] [49]。然而，Titans的局限在于它只有两个层级的参数更新：快速层（通常指模型常规部分）和慢速层（记忆模块），对应只能进行"一次"上下文内的参数调整 [32]。这意味着Titans虽然具备一定的**元学习**能力（在推理中学习记忆），但仍属于**一阶**的范畴。

HOPE 引入的 **自我修改**特性则突破了一阶限制，使模型拥有**多阶嵌套**的自适应能力 [33]。HOPE 的序列模型不仅可以**像Titans那样基于输入更新记忆**，更进一步，它**可以学习如何更新自身**——即**学习一个用于更新自己参数的算法** [50] [51]。这被称为**自引用学习过程**：模型在序列运行中，将自身作为学习对象，不断调整内部权重（尤其是记忆相关参数）以最小化某种内部目标（例如预测误差） [35] [36]。简单来说，HOPE在推理时相当于在内部**执行了一系列小的梯度下降**，用新数据优化部分模型参数，然后再用于下一时刻的预测 [31] [36]。例如，HOPE会让传统Transformer中固定的键/值投影矩阵在每个时间步根据当前上下文做出微调，从而实现**逐步的in-context adaptation** [52] [53]。通过这种方式，HOPE 获得了理论上**无界的上下文学习层级**：模型可以在推理阶段多次"自我微调"，不断地把新经验融入自身参数，实现真正的**终身学习** [54] [36]。

值得注意的是，实现自修改并不只是简单地在推理中继续训练模型（那样容易破坏已有能力），而是要**局部、受控**地更新特定记忆模块，同时保护其他部分不受干扰。这需要设计专门的内嵌优化规则或门控机制。NL论文附录中提出了一种基于梯度下降变体的方案，用于在模型内部执行参数更新 [55] [56]。总之，自修改模块赋予HOPE类似**元学习算法**的功能，使模型能够**自主调整**，这在持续学习场景下非常关键：模型可以**一边接受新任务输入，一边迅速调整部分参数以适应当前任务**，而整体架构通过多频率层次确保新知识不会完全覆盖旧知识 [15]。

多频率梯度更新机制

多频率更新是HOPE架构在训练和推理中的一项重要机制，贯穿于上述CMS和自修改设计中。简单来说，就是**不同的参数按照不同的时间步频率进行梯度更新** [7]。在CMS中，这体现在长程记忆模块的权重并非每个step都更新，而是隔一定步长才更新一次；而短程模块可以每步更新，从而捕获快速变化 [7]。这种**分层次更新**确保了模型的不同记忆层各司其职：**高频更新层**专注于近期模式学习，**低频更新层**负责累积长期稳定的知识 [7] [57]。

在训练过程中，多频率更新通常通过**梯度累积或参数冻结策略**实现。例如，可以将模型参数按层级分组：第1层参数每个batch都更新，第2层参数每隔5个batch更新一次，第3层参数每隔50个batch更新一次，等等（具体间隔可根据任务和模型规模设定）。这对应了 $C^{(ℓ)}$ 的概念：层级ℓ每经过 $C^{(ℓ)}$次迭代才执行一次优化步 [7]。这样做的效果是，**外层慢速模块在短期内看起来是"冻结"的**，它只偶尔调整，因而保留了对旧任务的记忆，不会被新数据的噪声频繁扰动；而内层模块快速收敛新模式，提供即时适应能力。当达到更新间隔时，再将慢速模块按照累积的误差信号进行更新，相当于将近期学到的新知识**整合压缩**进长期记忆中 [7]。这种机制类似于人类大脑中的**记忆巩固**：大脑在清醒时（对应内层模块）快速适应环境变化，但主要依赖短期记忆；在睡眠或休息时（对应外层模块

更新时机）才会将短期记忆重放并整合到长期记忆中。NL的连续记忆系统正是模拟了这样的过程，**将单一步长的训练扩展为多个时间尺度的协同训练** [7] [58]。

对HOPE而言，多频率梯度更新是实现其持续学习性能的核心：它保证了模型**既能及时学习新知识，又不会过快遗忘旧知识** [15]。工程上需要精心设计训练循环来支持这一点（下文将详细建议）。总之，多频率更新机制将传统"每步统一学习率更新所有参数"的范式转变为"**分层次分时更新**"，为模型提供了类似**短期记忆反复重写、长期记忆缓慢固化**的能力 [7] [40]。

nanochat 模型结构与 NL 改造分析

nanochat 仓库中的模型本质上是一个标准的GPT式**Transformer**网络 [59]。例如，其"d32"配置表示**32层Transformer 堆叠**、约19亿参数，采用~38B tokens的大规模语料预训练 [59]。模型使用多头自注意力（处理当前窗口上下文）和前馈网络（提取特征并存储知识）交替堆叠的结构，context length为1024，支持的词表大小为65536等 [60]。按照传统观点，**注意力层**充当模型的**短期工作记忆**，负责在每次推理时聚合当前上下文信息；而**前馈层权重**在预训练后基本固定下来，可以视为模型的**长期知识库**（长期记忆） [44]。训练nanochat时采用标准的反向传播和优化器（例如Adam或SGD），所有层参数一起以同频率迭代更新。

要将 nanochat 的模型结构迁移为NL/HOPE范式，需要对其现有架构引入多层次的记忆机制和嵌套优化过程。具体分析如下：

- **增加多层记忆模块**：按照连续记忆系统（CMS）的理念，nanochat的Transformer需要扩展出**多级别的记忆子结构**。目前每个Transformer层只有一个前馈模块充当隐含的长期记忆。可考虑在模型中**增设额外的记忆层或模块**，例如在原有前馈层之外，再串联一个或多个子层，使之形成**记忆链** [7]。这些新增的记忆子层将各自以**不同的更新频率**训练：靠近输入的一层可以频繁更新，越深层的记忆越延迟更新，从而分别存储不同时间跨度的信息 [7] [57]。在实现上，可以将这些子层设计为与前馈层类似的MLP，但参数更新由自定义逻辑控制（详见下节建议）。

- **引入嵌套优化器/深度优化器**：nanochat当前使用的优化器（假设为Adam或AdamW）可以替换或改造为NL框架下的**"深度优化器"** [16] [25]。这意味着优化器本身拥有**可学习的内部状态**（如前述带神经网络的动量项）而非固定公式。工程上，需定制一个优化器类：例如在优化步骤中，引入一个小型MLP根据当前梯度序列输出新的更新量（替代普通Adam的动量计算） [26]。这样，优化器就成为模型训练流程中的**嵌套记忆模块**，能够学习更复杂的更新规则，提高对梯度模式的适应性 [26] [27]。

- **参数更新频率分层**：在训练策略上，需要改变目前"每步更新所有参数"的做法，将不同模块设置不同的更新频率 [7]。可将nanochat模型的参数按功能/层次划分为若干组：例如，将快速变化的部分（如注意力层或新增的短期记忆模块）设为**高频更新组**，慢速演化的部分（如深层前馈长程记忆）设为**低频更新组**。然后修改训练循环，使高频组每个batch都梯度下降，而低频组隔N个batch才更新一次 [7]。这种分层更新需要精细控制，但能确保模型的"长期记忆"不会被新梯度反复覆盖，从而缓解灾难性遗忘，同时"短期记忆"又能紧跟最新数据变化 [7] [57]。

- **支持自修改前向过程**：Transformer 默认的前向传播是纯前馈计算，而 HOPE 要求模型在前向过程中能**动态调整自身**。为此，需要在 nanochat 模型中融入类似 RNN 或元学习的机制。例如，可以在每个时间步引入**内部学习步骤**：计算当前步预测误差或"惊讶度"，然后根据这一信号**即时更新**某些内部状态或参数，再用于下一步计算 [54]。具体做法可能是在模型中维护一个可训练的"快速权重"矩阵，初始为零，每来一个新输入就根据某种更新规则（如近似梯度下降或一个小网络映射）修改该快权重，从而影响下一token的输出。这

种**内嵌优化循环**使模型具有**上下文自适应**能力，相当于让nanochat具备了Titans/HOPE那样“边读边学”的本领。

总之，从架构上看，我们需要改造nanochat，使其从“固定层+固定优化器”的模式，升级为“**多层记忆+多级优化**”的模式。下一节将基于上述分析，提出具体的模块调整方案和代码实现建议。

模型改造方案与实例示范

针对以上分析，我们提出以下具体的nanochat模型改造方案，以迁移NL论文中的理论与HOPE架构：

1. **连续记忆模块的集成**：在nanochat的Transformer中，引入一个**“连续记忆系统”**组件。比如，可以创建一个类 `ContinuumMemoryModule`，内部含有若干层线性/MLP子模块（对应不同频率的记忆层）。将其插入Transformer的主干：例如每个原始Transformer Block的末端，用 `ContinuumMemoryModule` 取代原来的单一前馈层。这样，输入首先经过注意力层，得到的输出通过连续记忆模块的多层MLP依次变换，然后再进入下一Transformer层。需要特别实现**参数的分组及更新控制**（详见训练流程建议）：为连续记忆模块内的每一子层设定独立的参数组和更新周期 [7]。例如：`memory_layers[0]`每步训练都更新，`memory_layers[1]`每隔5步更新，`memory_layers[2]`每隔50步更新，依此类推。这样就实现了一个在代码层面**多层次记忆**的结构，各层通过序贯调用形成**长短结合**的记忆通道 [7] [45]。

2. **自适应优化器的替换**：将nanochat训练中使用的优化器替换为NL框架下的**深度优化器**实现。例如，可创建一个新的PyTorch优化器类如`DeepMemoryOptimizer`，其内部维护多个**可学习状态张量**。以模拟**深度动量梯度下降**为例：为每个待训练参数引入一个额外的“动量参数矩阵”M，用一个两层的小型网络来更新M [29]。更新规则可以参考NL论文的改进动量公式，例如：$M \leftarrow \alpha M - \eta \,\phi(G)$，其中$\phi(G)$是以当前梯度$G$为输入的MLP输出 [29]。最终参数更新量则用新的M值而非常规Adam公式计算。这要求编写自定义的`step()`函数，在里面执行上述神经网络运算。通过这种方式，优化器自身具备了**记忆先前梯度并自适应调整**的能力 [26] [24]。在实现中需注意高效性，可将历史梯度缓存并批量输入小网络计算更新，以减少开销。

3. **前向过程的嵌套更新**：修改nanochat模型的`forward`函数逻辑，使其支持**自修改**。一种可行方案是在模型中维护一些**快照参数/状态**，每次forward迭代时根据当前输入和先前输出误差对这些快照参数进行更新，然后再用于下次计算。例如，可以给Transformer每一层增加一个“快速偏置”参数ΔW，初始化为零张量。在处理一个新token时，先计算常规输出以及损失/置信度，然后以**未参与梯度图**的方式（detach当前计算图，以免干扰主梯度）根据损失对ΔW执行一次手工更新（模拟梯度下降）：$\Delta W \mathrel{+}= -\gamma \frac{\partial L}{\partial W}$（$\gamma$为内循环学习率）。更新后的$\Delta W$立即应用到模型权重$W$上，用以处理下一token。如此，每个时间步模型权重都会微调以适应最近输入造成的误差。这种手工更新可以通过PyTorch的`.detach()`和手动梯度计算（比如利用`torch.autograd.grad`）实现。更一般地，也可以训练一个**元网络**Φ，输入当前隐藏状态或误差信号，输出ΔW的增量，从而学习到比简单梯度更复杂的自修改规则 [54]。无论采用哪种方式，都应确保这些快照更新不会破坏全局参数，只影响局部行为——例如将$\Delta W`的值限制在一定范围，或只应用于特定层的投影矩阵（如仅动态调整注意力的Key/Value投影矩阵，这也是论文中提到HOPE**动态化K/V投影**的做法 [52] [53]）。

4. **模型结构配置调整**：在整合上述新组件后，需要根据HOPE架构的报告结果调整nanochat模型的超参数。例如HOPE论文中在760M和1.3B参数规模上获得了优于Transformer的效果 [61] [62]。因此在nanochat的改造中，可以尝试**增加一些参数开销用于记忆模块**（比如每层增加的MLP记忆链路可能需要一定维度扩张）并适

当减少原模型的某些部分参数，保持总参数量在可控范围内。还要为多频率更新引入新的超参数：如各记忆层的更新间隔$C^{(\ell)}$，内循环学习率γ等。这些参数可先依据论文建议设定初值，然后通过实验调整。

以下是一个简化的代码伪示例，将上述要点整合：

```
# 伪代码示意 nanochat 模型改造的一部分

class ContinuumMemoryModule(nn.Module):
    def __init__(self, input_dim, hidden_dim_list):
        super().__init__()
        # 根据隐藏维度列表创建多层MLP链
        self.mlps = nn.ModuleList([
            nn.Sequential(nn.Linear(d_in, d_out), nn.ReLU())
            for d_in, d_out in zip([input_dim]+hidden_dim_list[:-1],
hidden_dim_list)
        ])
        # 记录每层的更新步计数
        self.register_buffer('update_counters',
torch.zeros(len(hidden_dim_list)))
        # 设置每层的更新间隔，可以是超参数列表
        self.update_intervals = [1, 5, 50, ...]  # 例：不同层的更新频率

    def forward(self, x):
        out = x
        for i, mlp in enumerate(self.mlps):
            out = mlp(out)  # 前向通过MLP层
        return out

    def update_parameters(self, gradients):
        # 根据设定的频率间隔选择性更新各层参数
        for i, grad in enumerate(gradients):
            self.update_counters[i] += 1
            if self.update_counters[i] % self.update_intervals[i] == 0:
                # 只有满足更新间隔的层才执行参数梯度下降
                for param, g in zip(self.mlps[i].parameters(), grad):
                    param.data -= learning_rate * g
```

上述伪代码展示了一个连续记忆模块的大致结构：包含多层MLP，每层有自己的`update_counter`和`update_interval`来决定何时更新参数。实际实现中，我们会将各层参数的梯度累积在`gradients`中，在满足条件时才应用优化步骤。这段逻辑通常在训练循环中实现（例如通过自定义优化器或手动控制），因此接下来讨论训练流程重构。

训练流程的重构建议

改造nanochat为NL/HOPE范式后，其训练流程需要相应调整，以充分发挥新架构的优势。以下是针对**HOPE风格训练循环、多频率参数更新机制、记忆压缩与重放**的建议：

1. **多层次嵌套的训练循环**：传统训练是一个单一循环遍历数据并更新模型参数。针对引入的多频率参数组，应实现**嵌套的训练循环**或者在每次迭代中分别处理不同参数组的更新。例如：
2. 外层循环：遍历训练数据（epoch -> batch）。
3. 内层过程：对每个batch，先正常计算所有梯度，但**不要立即应用**。然后，对每个参数组检查其预定的更新间隔：
    - **高频组**（如快速记忆层、注意力层）：每batch都执行optimizer.step()更新。
    - **中频组**（如中等记忆层）：累积梯度，当达到设定批次数时再更新并清零累积。
    - **低频组**（如长期记忆层）：类似地，周期性地执行更新。

4. 这一逻辑可以通过PyTorch的**参数分组优化**实现：为各参数组设置不同的`optimizer`或调度器，在主训练循环中用条件语句调用不同组的`step()`[7]。也可以维护多个优化器实例，各自跟踪一部分参数。确保在skip更新时保留梯度以便后续step使用。

5. **自修改步骤与主训练的结合**：如果模型引入了前述**自修改快权重更新**，则训练时需考虑其影响。建议的训练过程是：

6. **前向传播**：对于每个训练样本序列，模拟推理过程，其中模型会多次调用自身（例如逐token地处理），并在内部执行若干自修改更新。要确保这些内部更新过程**不破坏**PyTorch的计算图，必要时可用`torch.no_grad()`或显式控制计算图范围。
7. **损失计算**：对整个序列输出计算总损失。
8. **后向传播**：反传时，模型参数的梯度将自然包括了自修改过程中产生的影响（如果自修改步骤本身是可微分设计的，比如使用元网络而非detach+手工更新)。若采用不可微的内部更新策略（如detach后的手工梯度步），则需要通过策略如**直通估计**(Straight-Through Estimator)或**附加损失**来训练这些内部过程。具体策略需根据实现方式确定。

9. **参数更新**：按前述多频率机制更新不同参数组。**注意**：自修改相关的参数（如元网络权重）本身也属于需要更新的参数组，通常应高频更新以及时适应。

10. **记忆压缩与巩固**：连续记忆系统引入后，模型拥有短中长不同层次的记忆模块。训练中应设计**记忆巩固/压缩**步骤，将短期记忆中有用的信息逐步融入长期记忆：

11. 可采用**周期性回放**策略：每当低频记忆模块准备更新时，从近期处理过的数据中选取具有代表性的一批（例如引起模型高损失的“惊奇”样本）重新输入模型（模拟“睡眠回放”），让模型巩固这些样本的梯度信号，然后再更新低频模块参数[7]。这样可以使长期记忆更有效地保存近期重要知识。
12. 或采用**蒸馏/压缩**策略：例如每隔固定步数，将高频模块的权重增量累积应用到低频模块上，类似软更新。例如：$W_{\text{low}} \leftarrow (1-\beta) W_{\text{low}} + \beta W_{\text{low}}^{*}$，其中*$W_{\text{low}}^{*}$*表示通过近期数据单独训练得到的低频权重近似值，$\beta$是很小的系数。这样低频权重慢慢向近期学习的最优值移动，但变化幅度很小，不会破坏旧知识。

13. **剪枝与压缩**：由于持续学习会让记忆模块累积大量信息，定期对记忆内容进行压缩也很重要。例如，可以限制低频模块的容量，对其权重进行正则化（如L_2正则）或使用矢量量化等技术压缩过去记忆。在代码实

现上，可在若干epoch后对长期记忆权重矩阵执行一次SVD截断或参数剪枝，以去除冗余信息，确保记忆模块高效 [63]。

14. **经验重放（Experience Replay）**：为进一步缓解灾难性遗忘，可在训练新任务或新数据时引入**replay buffer**机制 [64]。具体做法是在nanochat训练过程中维护一个有限的样本缓存，存储过去任务/数据中代表性的样本。当训练当前任务时，周期性地混入一些缓存样本进行训练，以**提醒模型不要遗忘**以前学到的知识。这种策略在NL框架下依然适用，并且可以和上述多频更新结合：例如，对低频记忆层的更新，使用一个特殊的小批量数据（包含新旧混合样本）来计算梯度，以实现对新旧知识的平衡更新。

15. **调试与验证**：引入上述改造后，训练过程会更复杂，需要仔细监控不同模块的行为。例如，记录各记忆层的梯度范数、更新频率与loss曲线的关系，验证高频层是否对新数据快速收敛、低频层是否稳定保持旧知识 [63]。可以设计一些**单步记忆测试**：让模型阅读一段文本并问题答复，观察在增加自修改和连续记忆后回答正确率是否提高，或长期一致性是否改善。也可以对模型进行**持续学习评测**：按照论文设置，让模型在多个顺序任务上训练，期间不使用旧数据，对比改造前后在早期任务上的性能下降幅度。如果HOPE改造正确，模型应表现出**更低的遗忘率**和**更高的长程推理能力** [15] [65]。

综上，经过这些训练流程改进，我们期望nanochat模型能够接近NL论文中HOPE架构的效果：在语言建模和常识推理任务上优于相同规模的标准Transformer，并且在长上下文处理和持续学习场景中展现出明显优势 [56] [66]。这些改造方案为后续工程实现提供了指导方向，开发者可以据此逐步将nanochat演进为一个支持自我优化、分层记忆且抗遗忘的更强大模型。最终，这将验证NL理论在实际大型语言模型上的价值，推动构建出更接近人类持续学习能力的AI系统 [67]。

参考文献：

- Ali Behrouz et al. *Nested Learning: The Illusion of Deep Learning Architectures*. NeurIPS 2025. [1] [16] [19] [20] [17] [26] [7] [30] [32] [33] [15] 等
- Google Research Blog: *Introducing Nested Learning: A new ML paradigm for continual learning* (Nov 2025) [44] [31] [54] [24] 等
- MarkTechPost: *Nested Learning…Views Models as Nested Optimization Problems* (Nov 8, 2025) [2] [5] [39] [7] [47] [33] [63] 等
- noaiLabs Medium: *Nested Learning: New Research from Google* (Nov 2025) [3] [25] [45] [68] 等
- nanochat 仓库 README 和 DeepWiki 文档 [59] [60]

---

[1] [11] [12] [16] [18] [50] [51] [52] [53] Nested Learning: The Illusion of Deep Learning Architectures | OpenReview
https://openreview.net/forum?id=nbMeRvNb7A

[2] [5] [6] [7] [8] [9] [15] [17] [23] [26] [28] [29] [30] [32] [33] [37] [38] [39] [40] [41] [42] [43] [46] [47] [58] [63] [65] Nested Learning: A New Machine Learning Approach for Continual Learning that Views Models as Nested Optimization Problems to Enhance Long Context Processing - MarkTechPost
https://www.marktechpost.com/2025/11/08/nested-learning-a-new-machine-learning-approach-for-continual-learning-that-views-models-as-nested-optimization-problems-to-enhance-long-context-processing/

[3] [4] [10] [13] [14] [25] [36] [45] [57] [64] [68] Nested Learning: New Research from Google | by noailabs | Nov, 2025 | Medium
https://noailabs.medium.com/nested-learning-new-research-from-google-6ae0ae38656f

[19] [20] [21] [22] [55] [56] [61] [62] [66] abehrouz.github.io
https://abehrouz.github.io/files/NL.pdf

[24] [27] [31] [34] [35] [44] [54] [67] Introducing Nested Learning: A new ML paradigm for continual learning
https://research.google/blog/introducing-nested-learning-a-new-ml-paradigm-for-continual-learning/

[48] [2501.00663] Titans: Learning to Memorize at Test Time - arXiv
https://arxiv.org/abs/2501.00663

[49] Titans: Where Tokens (Almost) Live Forever | by Ryan Partridge
https://medium.com/@achronus/titans-where-tokens-almost-live-forever-c995d9a84e0e

[59] GitHub - karpathy/nanochat: The best ChatGPT that $100 can buy.
https://github.com/karpathy/nanochat

[60] karpathy/nanochat | DeepWiki
https://deepwiki.com/karpathy/nanochat